

Day 16 — Events in SAP CPI

1. What Is an Event?

Event: whatever action needs to take place is an event. In SAP CPI, **Events decide what actions should be performed** — in particular, **when a process starts and when it ends** (as we saw on Day 9).

Palette → Events Tab (9 Events)

#	Event	One-Line Purpose
①	Start Message	Starts the main integration process when a message arrives from the source system
②	End Message	Ends the message at a particular step and sends data to the target (or back to the source)
③	Start Event	Starts a Local Integration Process
④	End Event	Ends message processing in a Local Integration Process
⑤	Error Start Event	Starts an Exception Subprocess
⑥	Error End Event	Ends an Exception Subprocess with an error (status Failed)
⑦	Escalation End Event	Ends the message with the separate status Escalated
⑧	Terminate Message	Terminates the message in the middle of a flow
⑨	Timer (Start Timer)	Starts an iFlow on a schedule (Day 9)

(Note: the numbers ①–⑨ above are the palette order; your notes number them slightly differently in the later pages, so use the names, not the numbers, when revising.)

2. Start Message Event

If you want to start the Integration Process (or the integration flow), use the **Start Message** event. It picks the message from the source system and triggers the iFlow.

- Without a **Start Message** event, an iFlow cannot start from an external system.
- **Example:** Source system (Postman) → iFlow → Start Message event. It is **mandatory** in such an iFlow — **without this event, the iFlow won't start**.
- This works **only when we receive the message from a source system**.
- So we keep the Start Message event in the **Main Integration Process**, because it is the first point that triggers the message.

Nuance: your notes say the triggering point of an iFlow is *always* the Start Message event. That's true for iFlows triggered by an **external sender**. But a **scheduled** iFlow starts with the **Timer (Start Timer)** event instead (Day 9) — it needs no sender at all. So: "external trigger → Start Message; scheduled trigger → Timer Start."

3. End Message Event

If we want to end the message at a particular step, we use the **End Message** event.

Basically, it is useful when you want to **send the data to the target system, or send a response back to the source system**. It **sends the data to the target system and ends the iFlow there**.

4. Start Event and End Event — Local Integration Process

Event	Used For
Start Event (circle)	If you want to start a Local Integration Process , you use the Start Event
End Event (circle)	If you want to end the message processing in a Local Integration Process , you use the End Event



Both are **mandatory** — every Local Integration Process needs a Start Event and an End Event.

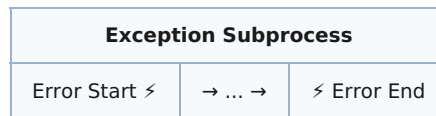
Why Can't We Use a Start Message Event in a Local Integration Process?

Answer: an **external call will hit only the Main Integration Process**, not the Local Integration Process. A Start Message event is what lets an outside sender connect — and a Local Integration Process is, by design, reachable only through a **Process Call** (Day 14).

5. Error Start Event & Error End Event — Exception Subprocess

Error Start Event. If you place an **Exception Subprocess** in any integration process, then throughout that integration process, **if any error occurs, that Exception Subprocess will be triggered**. *How* does this happen? **Inside the Exception Subprocess, the Error Start Event is used** — it is the entry point that catches the error.

Error End Event. Same as above — this is also used in the Exception Subprocess, to end it with an error.



Correction: your notes label this section "**Escalation Subprocess → Error Start Event**." There is no "Escalation Subprocess" in CPI — the correct name is **Exception Subprocess** (Days 9, 15). Escalation is a separate *end* event (section 6).

Added detail (verified): the Error End Event is not limited to Exception Subprocesses — SAP community guidance also uses it to end a **main or local process** with a failure (for example, to trigger a retry from a queue).

6. Escalation End Event

When to use it: if you want to represent the message status as "**Escalated**" — meaning the message is **not** an ordinary error (X), but you still want to show its status **differently** from Completed.

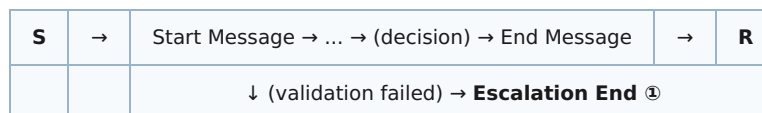
Example from Class

Employee information flows **Source → Target**. If the **Emp ID is missing (X)**, you **don't want to show that message as an error** — you want to show it as **escalated**. At that point, we use the **Escalation End Event**.

Real-Time Scenario

If payload validation fails, we can show the processed message status as **Escalated** — so support can filter Monitor for "Escalated" and see all business-data problems separately from technical failures.

Important Behaviour — Escalation End Does Not Call the Exception Subprocess



When the Escalation End is used internally (inside the normal flow), it does not call the Exception Subprocess — because no exception is raised; the message simply ends with status **Escalated**.

Verified: this matches SAP community documentation — an Escalation End Event placed in the normal flow raises no exception, so the Exception Subprocess is not triggered.

The "CamelHttpResponseCode = 400" Line in Your Notes

At the bottom of the page you wrote `CamelHttpResponseCode` — **Constant** — `400`. This is a Content Modifier header that sets the **HTTP status code the sender receives**.

Please verify this one with your trainer: according to SAP community articles and an SAP Knowledge Base article, when a flow **ends with an Error End or Escalation End event**, the sender **always gets a standard HTTP 500** with a generic message that includes the MPL ID — you **cannot** set a custom code like 400 in that case. A custom response code works when the flow ends with an **End Message** event. So `CamelHttpResponseCode = 400` belongs with **End Message** endings (see Section 8), not with Escalation End.

7. Terminate Message Event

If you want to terminate the message flow in between, we can use this event.

Your trainer called this the "**complexity part**": where exactly, and which event, can be used.

Which Event Goes Where — Summary Matrix

Event	Main Integration Process	Local Integration Process	Exception Subprocess
Start Message	Yes (start)	No	No
Timer Start	Yes (scheduled start)	No	No
End Message	Yes (end)	—	Yes (end; message becomes Completed)
Start Event	No	Yes (start)	No
End Event	—	Yes (end)	Yes (end; message becomes Completed)
Error Start Event	No	No	Yes (start)
Error End Event	Yes (ends with failure)	Yes (ends with failure)	Yes (end; message becomes Failed)
Escalation End Event	Yes	Yes	Yes (end; message becomes Escalated)
Terminate Message	In the middle of a flow	In the middle of a flow	—

How to use this table: it is built from your notes plus SAP documentation. Your trainer also gave you an "**events_process**" **Excel file — open Sheet 2** and observe where each event can be used (this was the class instruction). Treat the trainer's Excel as the authoritative class reference and compare it with this table; where a cell shows "—", I wasn't able to confirm it from your notes or SAP's documentation, so check the Excel for that cell.

8. Interview Perspective — Two Mandatory Questions

Your trainer flagged two questions that are **mandatory from an interview perspective**:

Question ① — What is the Escalation End Event, and why do we use it?

Model answer: The Escalation End Event ends the message with a separate status, **Escalated**, instead of Completed or Failed. We use it when a message is not a technical failure but needs attention — for example, a business validation failure (missing Emp ID). It lets support **filter Monitor for Escalated messages** separately from real errors, and — used in the normal flow — **it does not trigger the Exception Subprocess**.

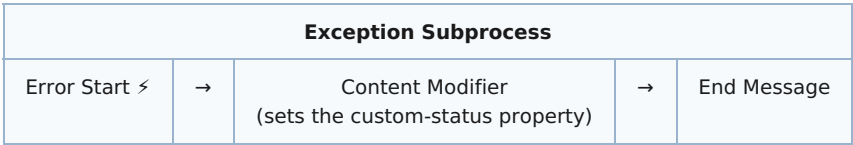
Question ② — What happens if we use End Message in the Exception Subprocess, and what happens if we use the Error End event?

Note on wording: your notes write "...and what will happen if we use *error start event*?" The Error **Start** event is just the entry of the Exception Subprocess — the meaningful comparison is with the Error **End** event, so I've answered it that way. If your trainer really meant Error Start, ask for clarification.

Exception Subprocess Ends With	Status in Monitor	What the Sender Gets
End Message	Completed — the error is treated as "handled"	A normal response — you can set a custom response and HTTP code (e.g., 400)
Error End Event	Failed	Standard HTTP 500 error with the MPL ID
Escalation End Event	Escalated	Standard HTTP 500 error

Showing "Failed" Even When the Message Ends as Completed — Custom Status

The problem with End Message is that the failed message looks **Completed** in Monitor. Your trainer's solution: set a **custom status** using the exchange property `SAP_MessageProcessingLogCustomStatus` .



Content Modifier settings (Exchange Property tab):

Action	Name	Source Type	Source Value
Create	SAP_MessageProcessingLogCustomStatus	Constant	Failed

Result: in the **Monitor tab** the standard message status stays **Completed**, but in the message log the **custom status shows "Failed"** — so support can still tell a handled error from a truly successful message.

Small fix: your notes write the Source Type as "(property)". For a fixed text like *Failed*, the Source Type should be **Constant** (as in Day 10). Also note: the custom status is shown **in addition to** the standard status — the main status remains Completed (verified in SAP community guidance).